파일 I/O · JSON · 정규표현식

Python File I/O · JSON · Regex

cchyun@gmail.com

Part 1. 파일 I/O + JSON 처리

로봇 개발에서 파일 I/O가 필요한 이유

- **로봇은 실시간 동작뿐 아니라 상태를 기록하고 외부 설정을 읽어오는 기능이 반드시 필요**
  - **센서 로그 저장**
    - IMU, 엔코더, 배터리 잔량 등의 텔레메트리 데이터를 저장
    - 성능 분석 및 하드웨어 결함 진단 시 활용
  - **설정 파일 (Configuration)**
    - 센서 보정값(Calibration offset), 네트워크 IP 주소를 외부 파일에 저장
    - 코드 수정 없이 로봇의 동작 환경을 변경 가능
  - **지도 데이터**
    - 자율 주행 로봇이 사용하는 점유 격자 지도(Occupancy Grid) 저장·불러오기

open() 함수 — 파일 열기 모드

- **파이썬에서 파일을 다루는 가장 기본적인 함수:** `open()`
  - `'r'` **(Read):** 읽기 전용, 파일이 없으면 오류 발생
  - `'w'` **(Write):** 쓰기 전용, 기존 내용 삭제 후 새로 작성, 없으면 신규 생성
  - `'a'` **(Append):** 파일 끝에 내용 추가, 기존 내용 유지
  - `'rb'`, `'wb'` **(Binary):** 이미지·센서 원시 데이터(Raw data) 처리
  - **기본 사용 예시 (권장되지 않음)**

```
f = open("sensor_log.txt", "w")
f.write("Sensor Data: 10.5")
f.close()  # 직접 닫아줘야 함 → 실수하기 쉬움
```

with 문을 써야 하는 이유 — 파일 핸들 누수 방지

- **파일을 열고 나면 반드시 `close()`를 호출해야 하지만, 예외 발생 시 잊기 쉬움**
  - **파일 핸들 누수 방지**
    - 파일을 닫지 않으면 시스템 자원이 계속 점유
    - 다른 프로그램의 파일 접근 차단 및 시스템 성능 저하 가능
  - **`with` 문: 자동 종료 보장**
    - 블록이 끝나는 즉시 파이썬이 **자동으로 파일을 안전하게 닫아줌**
    - 데이터 처리 도중 오류가 발생해도 파일을 확실히 닫음

```python
# 권장 방식: with 문 사용
with open("sensor_log.txt", "w", encoding="utf-8") as f:
    f.write("Sensor Data: 10.5")
# 블록을 벗어나는 순간 자동으로 f.close() 호출
```

JSON이 로봇 개발에서 자주 쓰이는 이유

- **JSON (JavaScript Object Notation): 가볍고 사람이 읽기 쉬운 데이터 형식**
  - **가독성**
    - 텍스트 형식이므로 메모장으로 직접 열어 설정값 수정 가능
  - **파이썬과의 호환성**
    - JSON 구조 ↔️ 파이썬의 **딕셔너리(Dictionary) 및 리스트(List)** 와 자연스럽게 매핑
    - 처리가 매우 직관적

```
{
    "robot_id": "Alpha-01",
    "wheel_diameter": 0.1,
    "sensors": ["lidar", "imu"]
}
```

  - **확장성**: 동일 내비게이션 SW에서 JSON 파일만 교체하면 다른 로봇에 바로 적용

JSON 처리 4가지 핵심 함수

- **파이썬 `json` 모듈의 핵심 함수 4가지**

함수	대상	설명
`json.dump()`	**파일**	파이썬 객체 → JSON 형식의 **파일**로 저장
`json.load()`	**파일**	JSON 형식의 **파일** → 파이썬 객체로 변환
`json.dumps()`	**문자열**	파이썬 객체 → JSON 형식의 **문자열**로 변환
`json.loads()`	**문자열**	JSON 형식의 **문자열** → 파이썬 객체로 변환

- **구분 핵심**: 끝에 `s`가 붙으면 **String(문자열)** 대상
  - `dump` / `load` → 파일 객체
  - `dumps` / `loads` → 문자열

센서 로그 데이터를 JSON으로 저장하고 불러오기 (1/2)

```python
import json
import datetime

# 1. 저장할 센서 데이터 준비
sensor_data = {
    "timestamp": str(datetime.datetime.now()),
    "robot_id": "Alpha-01",
    "lidar_samples": [1.2, 1.5, 0.8, 2.3, 1.1],
    "battery_level": 85.5,
    "status": "NORMAL"
}

# 2. JSON 파일로 저장하기 (json.dump)
try:
    with open("robot_logs.json", "w", encoding="utf-8") as f:
        # indent=4: 사람이 읽기 좋게 들여쓰기
        json.dump(sensor_data, f, indent=4)
    print("로그 저장 완료!")
except Exception as e:
    print(f"저장 중 오류 발생: {e}")
```

센서 로그 데이터를 JSON으로 저장하고 불러오기 (2/2)

```python
# 3. JSON 파일 불러오기 (json.load)
try:
    with open("robot_logs.json", "r", encoding="utf-8") as f:
        loaded_data = json.load(f)
        print(f"불러온 로봇 ID: {loaded_data['robot_id']}")
        print(f"배터리 상태: {loaded_data['battery_level']}%")
except FileNotFoundError:
    print("파일을 찾을 수 없습니다.")
```

- **실행 결과:**

```
로그 저장 완료!
불러온 로봇 ID: Alpha-01
배터리 상태: 85.5%
```

- **핵심 포인트:** 저장 시 `json.dump()`, 불러올 때 `json.load()`, 항상 `with` 문 사용

자주 발생하는 파일 I/O 오류와 해결법

- **로봇 소프트웨어는 하드웨어와 밀접하게 동작 → 예외 처리가 매우 중요**
  - `FileNotFoundError`: 지정한 경로에 파일이 없을 때
    - 해결: `os.path.exists()`로 확인 또는 `try-except`로 처리
  - `PermissionError`: 다른 프로그램이 파일을 열었거나 쓰기 권한 없음
    - 해결: 파일 사용 중 여부 확인, 로봇 시스템에서 `sudo` 권한 필요 여부 점검
  - `UnicodeDecodeError`: 파일 인코딩(UTF-8, CP949 등)과 파이썬 설정 불일치
    - 해결: `open()` 호출 시 `encoding="utf-8"` 옵션 명시
  - `JSONDecodeError`: JSON 문법 오류(괄호 누락 등)
    - 해결: 저장 파일 형식 확인, 빈 파일 여부 체크

예외 처리 — 안정적인 로봇 루프 유지

- 로봇 소프트웨어는 예상치 못한 상황에서도 멈추지 않아야 함

```python
import json, os

def load_robot_config(path):
    if not os.path.exists(path):
        print(f"[경고] 설정 파일 없음: {path}")
        return {}

    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError as e:
        print(f"[오류] JSON 형식 오류: {e}")
        return {}
    except PermissionError:
        print("[오류] 파일 접근 권한 없음")
        return {}

config = load_robot_config("robot_config.json")
```

Part 2. 정규표현식 (Regex)

정규표현식이 왜 필요한가 — 비정형 로그 파일 파싱

- **자율 주행 시스템은 초당 수십 건 이상의 비정형 로그 데이터를 생성**
  - **성능 진단**
    - 수천 줄의 로그 중 특정 시간의 `ERROR` 메시지만 골라내기
    - "모래사장에서 바늘 찾기" 작업을 자동화
  - **레거시 센서 데이터 처리**
    - GPS(NMEA 문자열), 직렬(Serial) 통신 센서 등 고유 형식의 문자열
    - 파이썬 딕셔너리와 같은 구조화된 데이터로 변환

```
[2026-04-07 10:00:12] ERROR: Motor torque limit exceeded (Value: 42.7)
[2026-04-07 10:00:15] INFO: Sensor update - Temp: 24.3, Humid: 40
```

- **정규표현식 없이 이 데이터를 파싱하면?** → 복잡한 문자열 분리 코드 필요

핵심 패턴 문법 — 메타 문자 (1/2)

- **정규표현식은 메타 문자(Metacharacters)를 사용하여 특정 규칙을 정의**
  - `.` : 줄바꿈을 제외한 **모든 한 문자**와 매치
  - `*` : 바로 앞 문자가 **0번 이상** 반복
  - `+` : 바로 앞 문자가 **1번 이상** 반복
  - `?` : 바로 앞 문자가 **0번 또는 1번** (옵션)
  - `[]` : 대괄호 안의 **문자들 중 하나**와 매치
    - 예: `[0-9]`는 모든 숫자, `[a-z]`는 소문자
  - `^` / `$` : 각각 문자열의 **시작**과 **끝**

핵심 패턴 문법 — 메타 문자 (2/2)

- **자주 사용하는 단축 표현(Shorthand)**

  - `\d` : 모든 **숫자**와 매치 (`[0-9]`와 동일)
  - `\w` : **알파벳, 숫자, 언더바(`_`)** 와 매치
  - `\s` : **공백 문자**(스페이스, 탭 등)와 매치

패턴	의미	예시 매치
`\d+`	1개 이상의 숫자	`42`, `100`
`\d+\.\d+`	실수 형태	`24.3`, `85.5`
`\w+`	단어 문자 1개 이상	`INFO`, `Alpha_01`
`.*?`	비탐욕적 임의 문자열	최소 매칭

주요 함수 차이와 사용법

- `re` 모듈의 4가지 핵심 함수
  - `re.match()` : 문자열의 **시작 부분**부터 패턴 일치 여부 확인
  - `re.search()` : 문자열 **전체**를 훑으며 패턴 일치 부분 탐색
  - `re.findall()` : 패턴과 일치하는 **모든 부분**을 리스트로 반환
  - `re.sub()` : 일치하는 패턴을 **다른 문자열로 교체**

```
import re

log = "[ERROR] Motor overheat at node_5"

re.match(r"ERROR", log)     # None (시작이 '[' 이므로)
re.search(r"ERROR", log)    # Match! (전체에서 탐색)
re.findall(r"\d+", log)     # ['5']
re.sub(r"ERROR", "WARN", log)  # "[WARN] Motor overheat at node_5"
```

그룹(Group)과 이름 붙은 캡처 활용법

- 괄호 `()`로 특정 부분을 그룹으로 묶어 나중에 따로 추출 가능
  - 일반 그룹: `(\d+)` → `.group(1)`로 접근
  - 이름 붙은 그룹 (Named Groups): `(?P<name>...)` → `.group('name')`으로 접근
    - 코드가 훨씬 직관적, 로봇 로그 분석에서 특히 유용

```python
import re

log = "[2026-04-07 10:00:12] ERROR: Motor torque exceeded"

# 이름 붙은 그룹으로 타임스탬프, 레벨, 메시지 캡처
pattern = r"\[(?P<timestamp>.*?)\] (?P<level>\w+): (?P<msg>.*)"
match = re.search(pattern, log)

if match:
    print(match.group('timestamp'))  # 2026-04-07 10:00:12
    print(match.group('level'))      # ERROR
    print(match.group('msg'))        # Motor torque exceeded
```

로봇 로그 파싱 예제 코드 (1/2)

```python
import re

# 실제 로봇 로그 예시 데이터
log_data = [
    "[2026-04-07 10:00:01] INFO: Battery level is 85.5%",
    "[2026-04-07 10:00:05] WARN: Lidar scan frequency dropped",
    "[2026-04-07 10:00:12] ERROR: Motor torque limit exceeded (Value: 42.7)",
    "[2026-04-07 10:00:15] INFO: Sensor update - Temp: 24.3, Humid: 40"
]

# 1. 타임스탬프 및 로그 레벨 추출 패턴
log_pattern = r"\[(?P<timestamp>.*?)\] (?P<level>\w+): (?P<msg>.*)"

for line in log_data:
    match = re.search(log_pattern, line)
    if match:
        # 2. 오류 레벨 필터링
        if match.group('level') == "ERROR":
            print(f"[긴급] 시간: {match.group('timestamp')}")
            print(f"       내용: {match.group('msg')}")
```

로봇 로그 파싱 예제 코드 (2/2)

```python
# 3. 특정 센서 값(실수) 추출
sensor_log = "Sensor update - Temp: 24.3, Humid: 40"

# 숫자.숫자 형태를 찾아 캡처
temp_match = re.search(r"Temp: (?P<temp>\d+\.\d+)", sensor_log)
if temp_match:
    print(f"현재 온도: {temp_match.group('temp')}°C")
```

- **실행 결과:**

```
[긴급] 시간: 2026-04-07 10:00:12
       내용: Motor torque limit exceeded (Value: 42.7)
현재 온도: 24.3°C
```

입문자가 자주 하는 실수

- **정규표현식을 처음 배울 때 흔히 겪는 함정 4가지**

  - `match()` **와** `search()` **의 혼동**
    - 문자열 중간 패턴 탐색에 `match()` 를 쓰면 아무것도 찾지 못함
    - 항상 `search()` **를 먼저** 고려할 것

  - **역슬래시 함정 (Backslash Plague)**
    - 파이썬 문자열과 Regex 모두 `\` 를 특수 문자로 사용 → 충돌
    - 반드시 **원시 문자열** `r"pattern"` 표기법 사용

```
# 잘못된 예: "\\d+" 처럼 이중 역슬래시 필요
re.search("\\d+", "42")

# 올바른 예: raw string으로 간결하게
re.search(r"\d+", "42")
```

입문자가 자주 하는 실수 — 탐욕적 매칭 & 메서드 선택

- **탐욕적(Greedy) 매칭 주의**
  - `.*`은 가능한 가장 긴 문자열을 먹어치움
  - 로그에서 `[시간] 메시지` 형태 파싱 시 `.*`을 쓰면 마지막 대괄호까지 포함될 수 있음
  - `.*?`**(비탐욕적, Non-greedy)** 수량자 사용 권장

```
import re
text = "[2026-04-07] [ERROR] Motor failed"

# 탐욕적: 처음 [ 부터 마지막 ] 까지 전부 매칭
re.search(r"\[.*\]", text).group()   # '[2026-04-07] [ERROR]'

# 비탐욕적: 최소 범위만 매칭
re.search(r"\[.*?\]", text).group()  # '[2026-04-07]'
```

- **문자열 메서드로 충분한 경우**
  - 단순 포함 여부: `re` 모듈보다 `in` 연산자 또는 `str.replace()`가 훨씬 빠름

실습 체크리스트

- **오늘 배운 내용을 로봇 프로젝트에 적용하기 위한 확인 사항** (1/2)
  - **가상 환경(venv) 설정**
    - 로봇 라이브러리(NumPy, PySerial 등) 간 충돌 방지를 위한 전용 가상 환경 생성·활성화
  - **안전한 파일 열기**
    - 파일 작업 시 예외 상황에서도 자동으로 닫아주는 `with` **구문** 사용
  - **JSON 데이터 검증**
    - 외부에서 로드한 `.json` 설정 파일의 딕셔너리 구조 확인

실습 체크리스트

- **오늘 배운 내용을 로봇 프로젝트에 적용하기 위한 확인 사항** (2/2)
  - **정규표현식 원시 문자열**
    - `\`가 포함된 Regex 패턴 작성 시 `r""` **(raw string)** 표기법 사용
  - **예외 처리 도입**
    - `FileNotFoundError`, `JSONDecodeError` 상황 대비 `try-except` **블록** 작성